



UNIVERSITÀ DI PISA

Fake news detection

Machine Learning & Data Mining

A.A. 2025-2026

Alessio Grillo – Andrea Giacomazzi

Problem

- **Information overload:** Rapid spread of unverified digital content.
- **Impact:** Polarization and manipulation of public opinion.
- **Scalability:** Manual fact-checking is too slow for modern data volumes.
- **Goal:** Need for automated, real-time detection systems.

Our proposal

- **Objective:** Develop a robust Machine Learning classifier to distinguish between real and fake information.
- **Approach:** Integrating Semantic Analysis with Stylometric Features.
- **Explainability:** Using XAI techniques (SHAP and LIME) to make model's decisions transparent.
- **Solution:** A functional application for real-time news assessment.

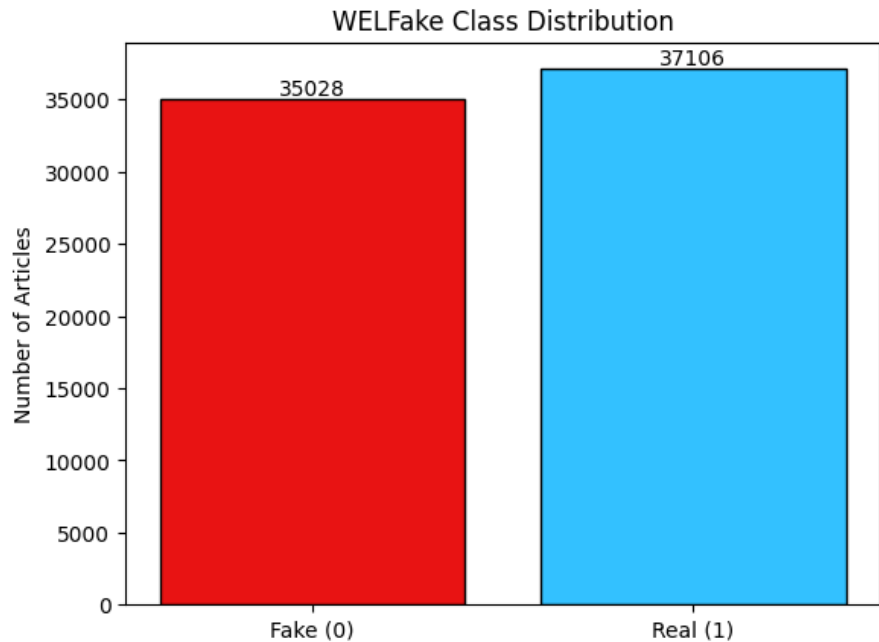
Dataset & EDA

- The dataset is **WELFake** and is present in Kaggle, it consists of 72,134 records, distributed across four primary features:

Feature	Type	Description
<i>Serial</i>	Integer	Unique identifier for each entry.
<i>Title</i>	String	The headline of the news article.
<i>Text</i>	String	The body content of the article.
<i>Label</i>	Binary	Target variable: 1 (Real) or 0 (Fake).

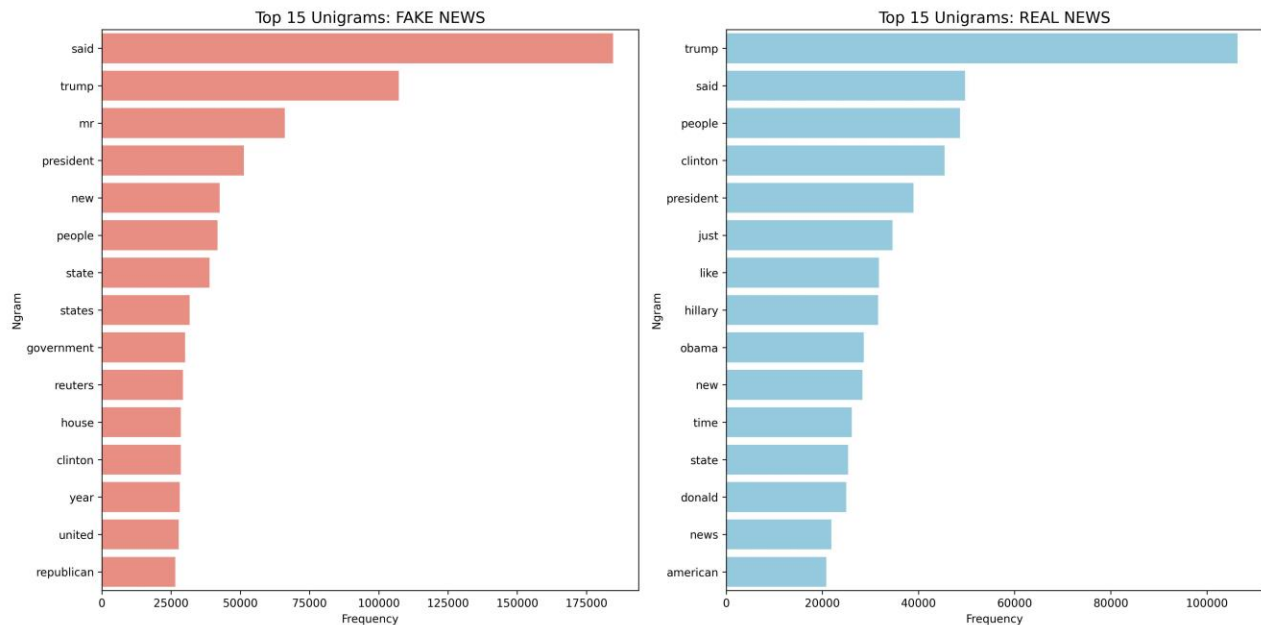
Dataset & EDA

- The WELFake dataset is remarkably well-balanced



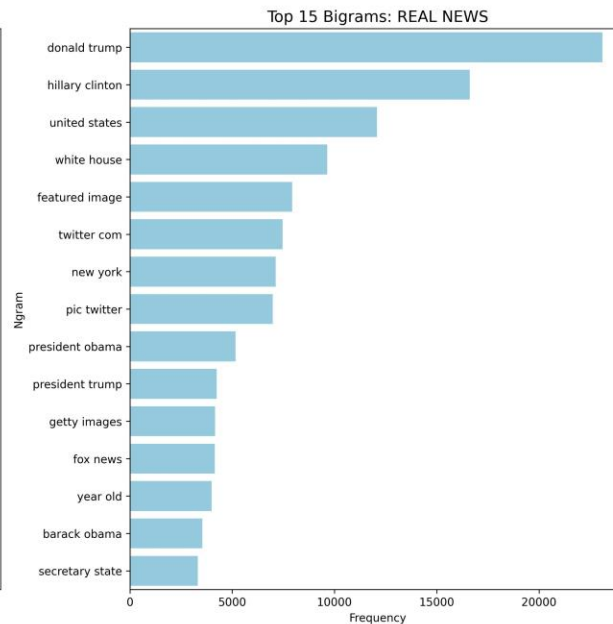
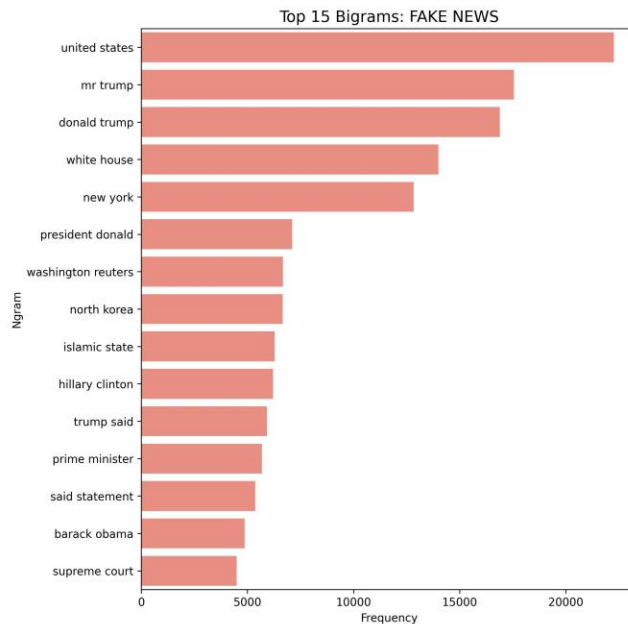
Dataset & EDA

- Unigram Frequency Analysis



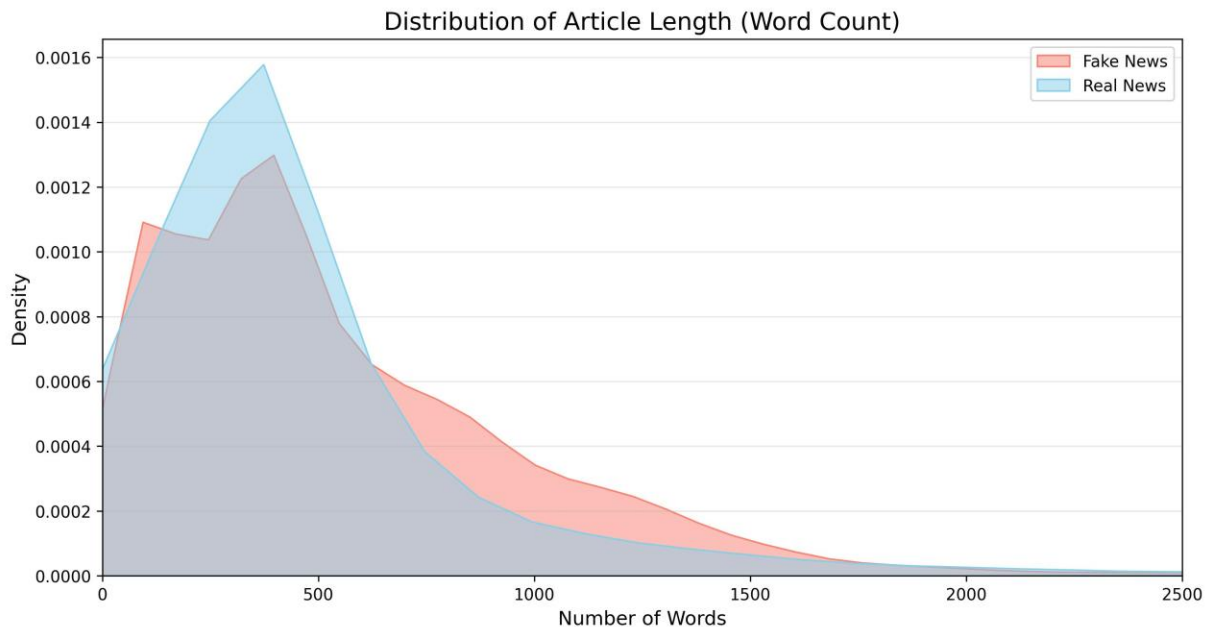
Dataset & EDA

- Bigram Frequency Analysis



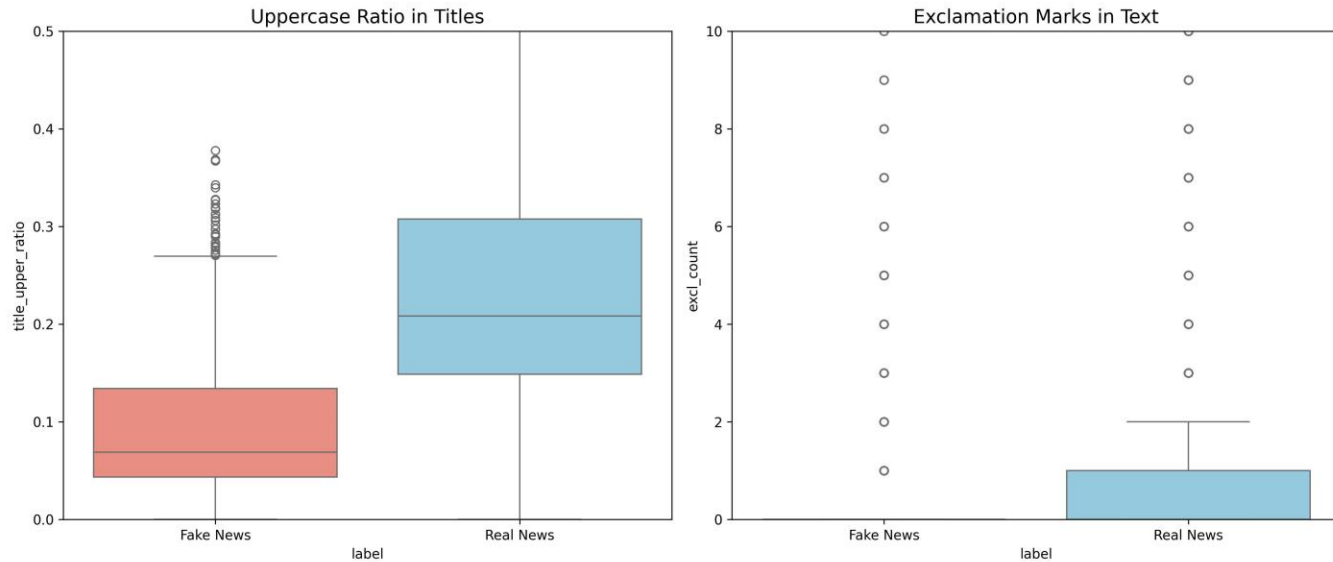
Dataset & EDA

- Distribution of article's body length



Dataset & EDA

- Uppercase ratio in titles and exclamation marks in article's body



Data preprocessing

Data Cleaning and Data Integration

- **Handling Missing Values:** Entries with null values in the title or text fields were identified. Given the size of the dataset, these rows were removed to maintain a high-quality corpus.
- **Feature Integration:** We performed a horizontal integration of the title and text columns. This creates a unified "content" feature, allowing the model to analyze headlines (often rich in clickbait patterns) alongside the full body of the article

Data preprocessing

Data Cleaning and Data Integration

```
# remove data with text null
df = df[["title", "text", "label"]].dropna(subset=['text'])
# substitute null values with "" in title column
df['title'] = df['title'].fillna("")
```

```
df['full_text'] = df['title'] + " " + df['text']
```

Data preprocessing

Stylometric Feature Engineering

- **Lexical Casing:** We calculated the percentage of uppercase words. High frequencies of capitalized text are a common proxy for sensationalism.
- **Article Volumetry:** We integrated the size of the article (total word count), as fake news often lacks the depth and length of authentic journalism.
- **Linguistic Integrity:** A dedicated module quantified the number of spelling errors, distinguishing professionally edited news from unverified content.
- **Punctuation Analysis:** We tracked the density of exclamation (!) and question marks (?) to detect inflammatory or non-neutral rhetoric.

Data preprocessing

Stylometric Feature Engineering

```
# add column with the length of the text
df['text_length'] = df['text'].apply(len)
# add column with ratio of exclamation point and question mark in the text
df['text_exclamation_ratio'] = df['text'].apply(lambda x: (x.count('!') + x.count('?')) / len(x) if len(x) > 0 else 0)
# add column with uppercase chars ratio in the title
df['title_uppercase_ratio'] = df['title'].apply(lambda x: sum(1 for c in x if c.isupper()) / len(x) if len(x) > 0 else 0)
# add column with ratio of exclamation point and question mark in the title
df['title_exclamation_ratio'] = df['title'].apply(lambda x: (x.count('!') + x.count('?')) / len(x) if len(x) > 0 else 0)
# add column with sum of exclamation point and question mark in the text
df['text_exclamation_count'] = df['text'].apply(lambda x: x.count('!') + x.count('?'))
```

```
spell = SpellChecker()

def count_spelling_errors(text):
    if not isinstance(text, str) or len(text) == 0:
        return 0
    words = re.findall(r'\b\w+\b', text.lower())
    if len(words) == 0:
        return 0

    misspelled = spell.unknown(words)
    return len(misspelled) / len(words)

print("Calculating...")
df['spelling_errors_ratio'] = df['full_text'].apply(count_spelling_errors)
```

Data preprocessing

Text Normalization, Tokenization, and Leak-age Prevention

- **Noise Removal:** We stripped non-alphanumeric characters using regular expressions.
- **Case Folding:** All text was converted to lowercase to unify the vocabulary.
- **Tokenization:** The clean text was segmented into individual tokens (words), which serve as the fundamental units for the subsequent vectorization.
- **Lemmatization:** To group inflected forms of words, tokens were reduced to their morphological roots (e.g., "running" to "run"), thereby reducing the dimensionality of the feature space without losing semantic meaning.
- **Custom Stopword Elimination:** Beyond standard English stopwords (e.g., "of", "the", "in"), we implemented a targeted exclusion list based on our exploratory N-gram analysis.

Data preprocessing

Text Normalization, Tokenization, and Leak-age Prevention

```
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')

nltk.download('wordnet')
nltk.download('omw-1.4')

stop_words = set(stopwords.words('english'))

lemmatizer = WordNetLemmatizer()

custom_stopwords = [
    'said', 'mr', 'mrs', 'just', 'like', 'new', 'year', 'time', 'also', 'would', 'one',
    'trump', 'clinton', 'hillary', 'obama', 'donald', 'barack', 'president',
    'state', 'states', 'government', 'house', 'white', 'republican', 'democrat', 'american',
    'reuters', 'washington', 'featured', 'image', 'images', 'getty', 'pic',
    'twitter', 'com', 'via', 'fox', 'news', 'video', 'youtube'
]
stop_words.update(custom_stopwords)
```

```
(method) def words(
    fileids: Any | None = None,
    ignore_lines_startswith: str = "\n"
) -> list[str]
```

Data preprocessing

Text Normalization, Tokenization, and Leak-age Prevention

```
def clean_text(text):
    text = re.sub(r'<.*?>', '', text)
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)

    text = text.lower()

    tokens = word_tokenize(text)

    lemmas = [lemmatizer.lemmatize(token) for token in tokens]

    cleaned_tokens = []
    for w in lemmas:
        clean_w = re.sub(r'^\w\s$', '', w)
        if clean_w and clean_w not in stop_words:
            cleaned_tokens.append(clean_w)

    return " ".join(cleaned_tokens)
```

Data preprocessing

Vectorization: 10,000 TF-IDF Features

- **Configuration:** TF-IDF using 10,000 features (Unigrams & Bigrams).
- **Dimensionality Control:**
 - Mitigates the "Curse of Dimensionality".
 - Optimizes the data-to-feature ratio for 72,095 samples.
 - Prevents statistical sparsity and improves class separability.
- **Noise Reduction:** Focuses on high-signal linguistic patterns to minimize overfitting.
- **Experimental Integrity:**
 - Vectorizer fitted exclusively on training folds.
 - Zero-knowledge approach for validation/test sets to prevent Data Leakage.

Data preprocessing

Feature Integration & Scaling

- **Objective:** Harmonize stylistic features (e.g., text length, punctuation) with TF-IDF vectors.
- **Normalization:**
 - Applied MinMaxScaler to map all features to a fixed $[0, 1]$ range.
 - Prevents high-magnitude variables (character counts) from biasing the model.
 - Aligns stylistic data with the inherently normalized TF-IDF scale.
- **Methodological Rigor:**
 - Followed a "Zero-Knowledge" approach.
 - Scaling parameters (min, max) were fitted only on training folds to prevent data leakage.
- **Feature Fusion:** Horizontal concatenation of the 5D stylistic vector and the sparse TF-IDF matrix to create the final feature set.

Data preprocessing

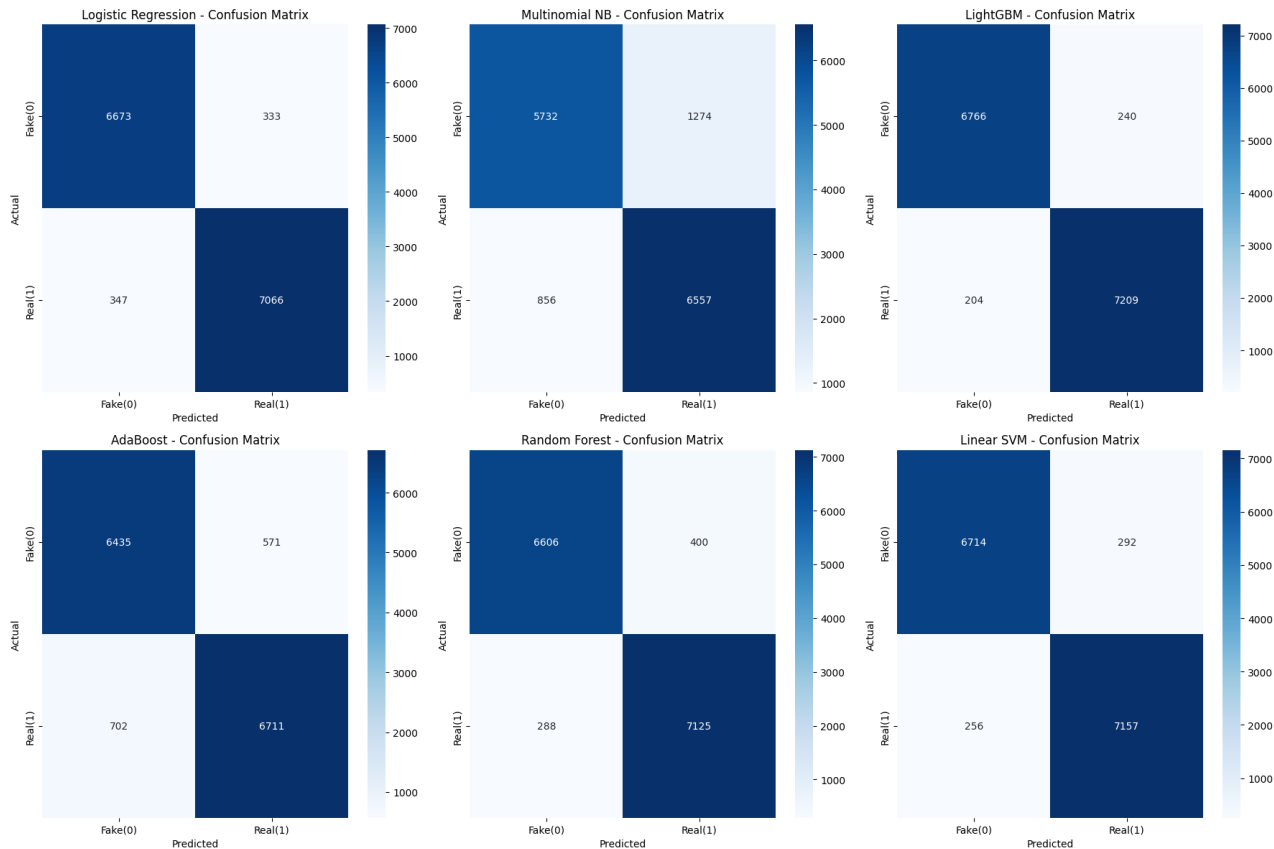
Vectorization & Feature Integration & Scaling

```
X_stylometric = df[['text_length', 'title_uppercase_ratio',  
                    'title_exclamation_ratio', 'text_exclamation_ratio',  
                    'spelling_errors_ratio']]  
X_text = df['cleaned_text']  
y = df['label']  
  
X_text_train, X_text_test, X_stylo_train, X_stylo_test, y_train, y_test = train_test_split(  
    X_text, X_stylometric, y, test_size=0.2, random_state=42, stratify=y  
)  
  
tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1,2))  
X_tfidf_train = tfidf.fit_transform(X_text_train)  
X_tfidf_test = tfidf.transform(X_text_test)  
  
scaler = MinMaxScaler()  
X_stylo_scaled_train = scaler.fit_transform(X_stylo_train)  
X_stylo_scaled_test = scaler.transform(X_stylo_test)  
  
X_train_final = hstack([X_tfidf_train, X_stylo_scaled_train])  
X_test_final = hstack([X_tfidf_test, X_stylo_scaled_test])
```

Models evaluation

- **Validation Strategy:** Stratified 10-Fold Cross-Validation.
- **Algorithms Tested:** Logistic Regression, SVM, Random Forest, AdaBoost, LightGBM.
- **Winner:** LightGBM (F1-Score: 0.971, Accuracy: 0.970).
- **Why LightGBM?** Best balance between speed, precision, and handling of our high-dimensional features (TF-IDF).

Models evaluation



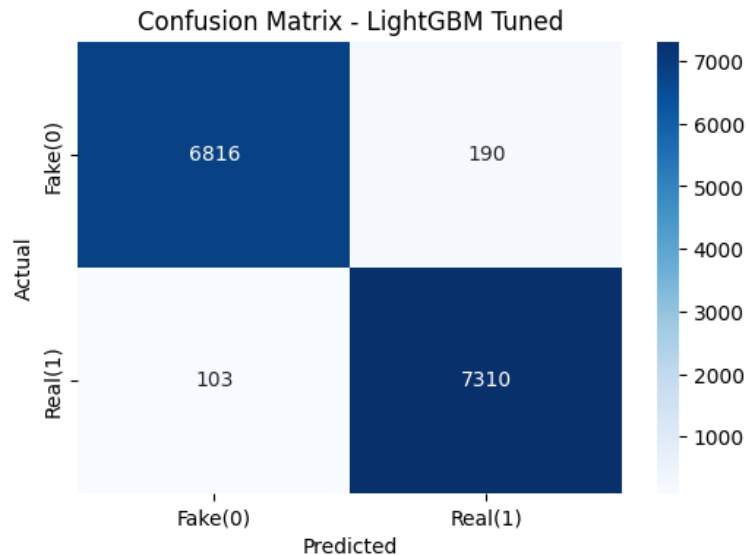
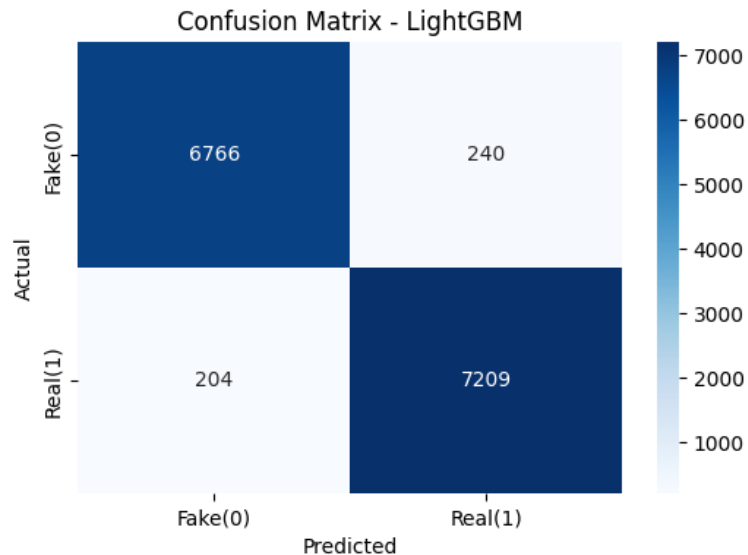
Models evaluation

Metric/Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
LightGBM	0.9705	0.9707	0.9719	0.9713	0.9957
Linear SVM	0.9646	0.9651	0.9660	0.9656	0.9934
Random Forest	0.9528	0.9503	0.9584	0.9543	0.9909
Logistic Regression	0.9530	0.9578	0.9504	0.9541	0.9908
AdaBoost	0.9075	0.9287	0.8883	0.9080	0.9705
Multinomial Naive Bayes	0.8534	0.8380	0.8863	0.8615	0.9282

LightGBM optimization

- **Technique:** Grid Search with Stratified Cross-Validation.
- **Target Metric:** Maximizing F1-Score.
- **Tuned Parameters:** learning_rate, num_leaves, max_depth, n_estimators.
- **Final Result:** F1-Score increased to 97.9% (Accuracy: 97.9%).
- **Key Benefit:** Better generalization and lower risk of overfitting.

LightGBM optimization



LightGBM optimization

Before Hyperparameter Tuning (Test set)

Metric	Value
Accuracy	0.9692
Precision	0.9692
Recall	0.9691
F1-Score	0.9691
AUC-ROC	0.9955

After Hyperparameter Tuning (Test set)

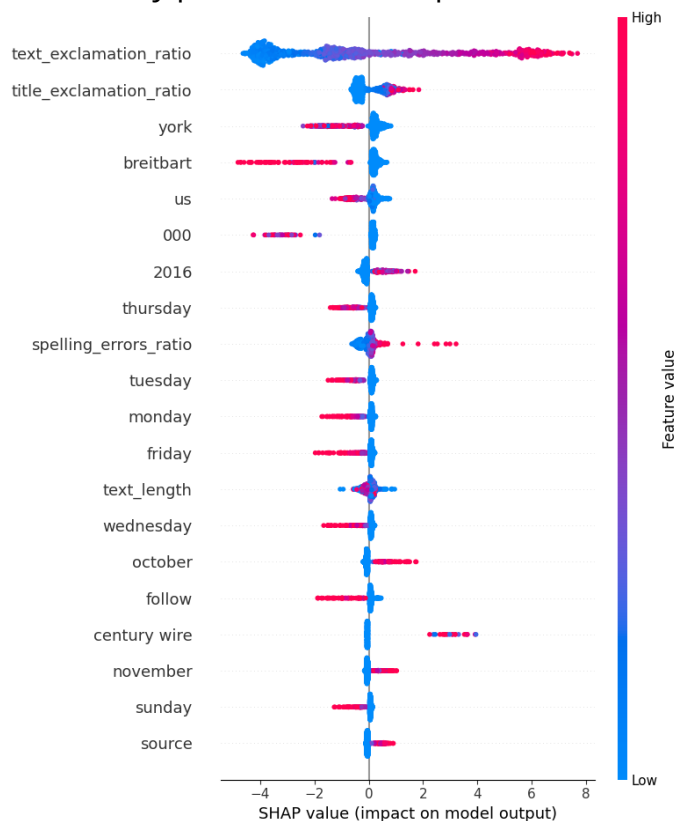
Metric	Value
Accuracy	0.9797
Precision	0.9799
Recall	0.9795
F1-Score	0.9797
AUC-ROC	0.9976

Model explainability

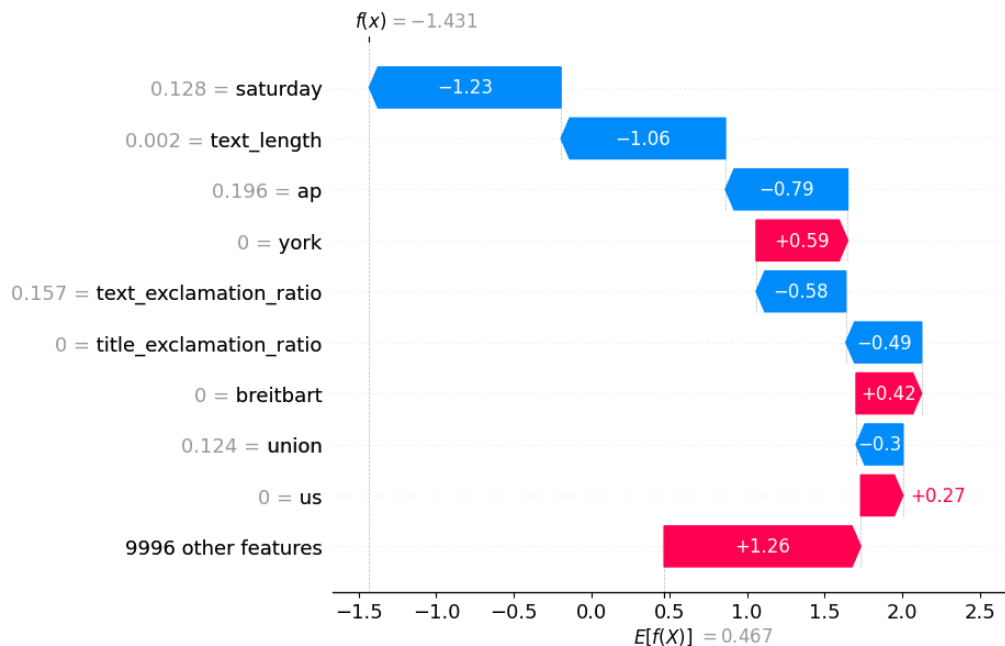
- **The "Black Box" Problem:** High performance requires transparency and trust.
- **Global Interpretation (SHAP):** Identifies the most impactful features across the entire dataset (e.g., specific keywords, exclamation ratio).
- **Local Interpretation (LIME):** Explains single, real-time predictions to the end-user.
- **Our Strategy:** SHAP for model validation, LIME for user-facing transparency.

Model explainability

SHAP summary plot – 20 most important features



Why did the model classify article 1 like this?



Our App

- **End-to-End Pipeline:** Real-time inference using the optimized LightGBM model.
- **Dual Input Architecture:**
 - **Online Mode (URL):** Automated web scraping to extract title and body from live articles.
 - **Offline Mode (Manual):** Direct raw text input for unpublished or custom content..
- **Transparent Output:** Automated generation of an interactive **LIME Report** for the end-user.

Conclusion and future works

- **Mission Accomplished:** High-performance hybrid model (97.9% F1-Score).
- **Transparency:** Successfully bridged the gap between complex ML and user trust (Explainable AI).
- **Ready-to-Use:** Delivered a functional, real-time Fake News Detector App.
- **Advanced Deep Learning:** Transition to Transformer-based models (e.g., BERT / RoBERTa).
- **Bias Mitigation:** Expand dataset to reduce US-politics domain bias.
- **Multilingual Support:** Extend detection capabilities beyond the English language.